Closing the agent execution gap: from 58.5% to reliable deploy

Your agent's core failure mode — excelling at planning and code generation while skipping `make test-backend`, `railway up`, and smoke tests — is the **single most documented weakness** across every autonomous coding tool today. The 9.3-point delta between skill-guided and baseline runs tells you the skills improve *what* code gets written, but don't solve the deeper architectural problem: nothing in your current loop *forces* execution. The good news is that SWE-Agent, Devin, Aider, and Claude Code have each solved this problem with different patterns, and the most effective approaches combine **structural enforcement** (hooks, state gates, API-level tool forcing) with **prompt-level execution mandates**. Below is a synthesis of concrete, copy-pasteable patterns drawn from Princeton's SWE-Agent architecture, Cognition's Devin 2.2 autofix loop, Aider's auto-test hooks, Anthropic's own Claude Code internals, and the latest eval research from SWE-bench Pro and Terminal-Bench.

Why agents stop at "here's the code" and how to fix it structurally

The root cause is statistical, not architectural. LLMs are trained overwhelmingly on text responses — writing code blocks and describing next steps is the path of least resistance. Without explicit forcing, the agent treats "here is the code" as a complete response. Research from "Can Tool-Augmented LLMs Be Aware of Incomplete Conditions?" confirms that **LLMs rarely pause to validate**; they force solutions by making assumptions. Context window pressure compounds this: as context fills, agents become less likely to make additional tool calls and instead "satisfice" with text. Cognition arXiv

The tools that solve this use layered enforcement. **SWE-Agent** gives the agent *no interface except bash* — it literally cannot do anything without executing shell commands. Every action is a `subprocess.run` call inside a Docker container. GitHub Vals AI There is no "write code only" path. **Devin 2.2** added computer use for self-verification: it launches and tests applications on its own Linux desktop, sends back screen recordings, and autofixes CI/CD failures automatically. Cognition When GitHub bots comment on PRs (linter, CI, security scanner), Devin picks up and fixes issues in a tight loop. Cognition **Aider** takes a hook-based approach: `--auto-test` runs the test suite after every AI edit, and non-zero exit codes trigger automatic fix attempts. This is deterministic — the agent cannot skip it.

Claude Code's architecture is the most relevant to your setup. It runs a single-threaded master loop: `while(tool_call) → execute tool → feed results → repeat`. arXiv When Claude produces a plain text response without tool calls, the loop terminates. Claude +2 The critical forcing mechanism is **hooks** — deterministic scripts that run at specific lifecycle points:

```
json
```

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "command": "make test-backend 2>&1 | tail -30"
      }
    ]
  }
}
```

This automatically runs your test suite after every file edit, injecting output into Claude's context. Unlike CLAUDE.md instructions (which are advisory and sometimes ignored), **hooks are deterministic** humanlayer — they execute regardless of what the model decides. Claude This is the single highest-leverage change you can make to your Winston workflow.

The five-layer execution enforcement stack

No single technique reliably forces execution. The most effective approach, drawn from across the research, layers five mechanisms from the API level up to the prompt level. Each layer catches failures the others miss.

Layer 1: API-level tool forcing. Claude's API supports `tool_choice: "required"`, which prevents text-only responses — the model *must* invoke a tool. For your skill files that handle the test/deploy phases, this eliminates the most common failure mode entirely. You can also force a specific tool: `tool_choice: {"type": "tool", "name": "bash"}` to ensure the agent executes a command rather than describing one. Claude API Docs

Layer 2: Deterministic hooks. Configure PostToolUse hooks in `.claude/settings.json` to auto-run `make test-backend` and `make test-frontend` after every Edit or Write operation. The test output flows back into context, giving the agent immediate feedback. This mirrors Aider's `--auto-test` pattern, DeployHQ which is enabled by default in their most effective configurations and is the reason Aider achieves strong retry improvement rates (pass_rate_1 of 57% jumping to pass_rate_2 of 77% after seeing test output).

Layer 3: State machine gates in skill files. Structure your skills as explicit state machines where the agent cannot transition to the next phase without meeting prerequisites:

```markdown
```

```
## Workflow States (MANDATORY — follow in order)

### STATE: implementing
- Valid transition → testing
- REQUIRES: code written to actual files (not just displayed)

### STATE: testing
- Valid transition → deploying
- REQUIRES: `make test-backend` exit code 0 AND `make test-frontend` exit code 0
- If tests fail: remain in this state, fix, re-test

### STATE: deploying
- Valid transition → verifying
- REQUIRES: `railway up` or `git push` executed with output captured

### STATE: verifying
- Valid transition → complete
- REQUIRES: smoke test HTTP 200 from deployed endpoint

You CANNOT reach "complete" without passing through every state.
A response without terminal output from the testing state is INCOMPLETE.
```

Layer 4: Proof-of-execution blocks. Require the agent to produce structured evidence for every change:

```markdown
After every code change, produce this block:

## Execution Evidence
- **Command run:** `[exact command]`
- **Exit code:** [0 or error code]
- **Output (last 30 lines):** [paste actual output]
- **Tests passing:** [YES/NO with count]
- **Smoke test:** [PASS/FAIL with endpoint and status code]

A response without this block is INCOMPLETE. "Tests should pass" is NOT acceptable —
you MUST run them and paste the output.
```

Layer 5: Anti-pattern bans. Explicitly ban the failure modes you observe most:

```markdown
```

```
## Banned Patterns
- Writing a code block without executing it when you have shell access
- Saying "the tests should pass" without running them
- Describing a deployment without running the deploy command
- Using "would," "could," or "should" in completion statements
- Showing terminal commands without executing them

Replace "this would work" with "I ran this and the output was..."
```

The research from QuantumBlack/McKinsey's production agentic systems confirms that **agents are good at generating content within bounded problems but struggle with meta-level workflow sequencing**. The solution is a conventional, rule-based workflow engine (your state machine gates) that enforces phase transitions, Medium not asking the LLM to self-regulate.

Structuring skill files for a three-runtime monorepo

Your Winston architecture — Next.js 14 frontend, FastAPI BOS backend, FastAPI Demo Lab backend with two API patterns — creates the classic monorepo routing problem. The agent needs to know which service owns a feature, what stack conventions apply, and where cross-cutting changes go. The dominant pattern across Cursor, Claude Code, Cline, and Aider is a **root-level router file plus nested service-specific instructions**.

For Claude Code, the hierarchy loads automatically: `~/.claude/CLAUDE.md` (global) → `project/CLAUDE.md` (root) → `project/apps/bos/CLAUDE.md` (service-level) → `.claude/skills/` (workflow-specific). A critical finding from HumanLayer's research: Claude Code's system prompt already contains **~50 instructions**, and frontier models reliably follow only **150-200 total instructions**. This means your CLAUDE.md should contain fewer than 60 lines humanlayer — ideally just a map and pointers. Humanlayer promptlayer

Here's the recommended structure for Winston:

```markdown
```

```
# CLAUDE.md (root — under 60 lines)

## Architecture Map
This monorepo has 3 runtimes and 2 API patterns:

### apps/frontend (Next.js 14)
- **Runtime**: Node.js / Next.js 14 App Router
- **Port**: 3000
- **Owns**: All UI, pages, client-side state
- **API pattern**: REST fetch from server components → BOS backend
- **Test**: `make test-frontend`
- **Deploy**: Vercel git push

### apps/bos (FastAPI BOS Backend)
- **Runtime**: Python 3.12 / FastAPI
- **Port**: 8000
- **Owns**: Business logic, database, auth, core API
- **Test**: `make test-backend`
- **Deploy**: `railway up`

### apps/demo-lab (FastAPI Demo Lab)
- **Runtime**: Python 3.12 / FastAPI
- **Port**: 8001
- **Owns**: Interactive demos, sandboxed experiments
- **Test**: `make test-demo`

## Service Boundaries (CRITICAL)
- Frontend code lives ONLY in `apps/frontend/`
- BOS backend code lives ONLY in `apps/bos/`
- Demo lab code lives ONLY in `apps/demo-lab/`
- NEVER import Python modules across apps/bos and apps/demo-lab
- Frontend communicates with BOS ONLY via REST API calls
- Shared types go in `packages/shared/`

## Execution Rules
- ALWAYS run tests after changes: the relevant `make test-*` command
- ALWAYS include actual terminal output in your response
- NEVER say "tests should pass" — RUN THEM
- See `agent_docs/` for detailed patterns per service
```

The word **"ONLY"** is a critical routing directive (Factory) — research from Factory.ai and Cursor community forums shows this prevents agents from placing code in the wrong service. The **Type/Stack/Role** format per service (visible in the Cursor monorepo pattern) gives the agent minimum viable routing information. (cursor)

For deeper context, use **progressive disclosure** — point to `agent_docs/service_architecture.md` rather than embedding everything in CLAUDE.md. (Humanlayer +2) An ETH Zurich study found that auto-generated CLAUDE.md files (via `/init`) actually make performance worse because they repeat information Claude can already discover by reading code. (DEV Community)

For Cursor compatibility, create parallel `.cursor/rules/*.mdc` files with glob-scoped rules: (Cursor)

```mdc
---
description: BOS Backend development rules
globs: "apps/bos/**/*"
alwaysApply: false
---
- FastAPI with Pydantic models for validation
- SQLAlchemy for database access
- Use dependency injection patterns
- Test with pytest: `make test-backend`
```

Redesigning your eval harness to measure execution completeness

Your current eval compares skill-guided (58.5%) vs baseline (49.2%), but if the grading rubric only checks whether code was written correctly, it misses the entire execution gap. The most important shift is moving from **binary code-quality scoring** to **multi-checkpoint execution scoring** inspired by TheAgentCompany and AgentBoard.

SWE-bench uses a dual-test methodology: **FAIL_TO_PASS** tests (confirming the issue is resolved) and **PASS_TO_PASS** tests (confirming no regressions). Both must pass for a task to be marked resolved. (arXiv +2) But this still only measures code correctness, not execution completeness. SWE-bench Pro (Scale AI, 2025) added longer-horizon tasks averaging **107.4 lines across 4.1 files**, and frontier models score only ~23% (Scale) — revealing the gap between simple patches and realistic feature work.

The benchmark most relevant to your problem is **Terminal-Bench 2.0** (Stanford/Laude Institute), which evaluates agents on real-world CLI tasks: building kernels, training ML models, configuring servers. (Tessl) Each task has containerized environments with verification scripts that check multiple output conditions. (Artificial Analysis) Frontier models score under 65%, capturing exactly the gap between "talking about doing" and "actually doing." (Hugging Face) (Tessl)

Here's a concrete multi-checkpoint rubric designed for Winston's eval harness:

```yaml
```

```yaml
execution_completeness_rubric:
  checkpoints:
    - id: code_written
      weight: 0.10
      grader: deterministic
      check: "git diff is non-empty for relevant service files"

    - id: code_valid
      weight: 0.10
      grader: static_analysis
      check: "ruff check (Python) / eslint (TS) pass without errors"

    - id: tests_invoked
      weight: 0.20
      grader: transcript_analysis
      check: "make test-backend or make test-frontend appears in command history"

    - id: tests_pass
      weight: 0.25
      grader: deterministic
      check: "test command exit code is 0"

    - id: deploy_executed
      weight: 0.15
      grader: transcript_analysis
      check: "railway up or git push appears in command history"

    - id: smoke_test_pass
      weight: 0.20
      grader: deterministic
      check: "HTTP GET to deployed endpoint returns 200"

  scoring:
    partial: sum(checkpoint_weight × checkpoint_pass)
    completion_bonus: 0.5 if ALL checkpoints pass
    planning_penalty: -0.30 if transcript shows reasoning but ZERO tool calls

  anti_planning_detection:
    trigger: "agent produced >500 tokens of text but executed 0 shell commands"
    action: "flag as planning-only failure, apply penalty"
```

The **planning penalty** is the key innovation. AgentBoard (NeurIPS 2024) introduced a "Progress Rate" metric that decomposes tasks into subgoals and measures incremental progress (Emergent Mind) — differentiating near-

complete tasks from minimal execution. NeurIPS arXiv Their finding: **open-weight models stop making progress after ~6 steps** in long interactions, exactly the pattern you're seeing where the agent plans extensively then stops.

For your 58.5% vs 49.2% comparison, the most revealing analysis isn't aggregate pass rate but **per-checkpoint pass rates**. If skill-guided runs score 90% on `code_written` but only 20% on `tests_invoked`, you know exactly where the bottleneck is. Aider tracks a similar metric: the delta between `pass_rate_1` (first attempt) and `pass_rate_2` (after retry with test feedback), which directly captures self-correction ability. Epoch AI Top agents on Aider's benchmark achieve 92.9% by enforcing test execution at least once per task. Refact.ai

---

The two-agent pattern Anthropic recommends for long-running tasks

Anthropic's engineering blog ("Effective Harnesses for Long-Running Agents," November 2025) describes a production-tested pattern directly applicable to Winston:

Initializer agent sets up the environment: runs `init.sh`, creates a `feature_list.json` with all features marked as failing, makes an initial git commit, and starts the dev server. **Coding agent** makes incremental progress per session: reads the progress file and git log, works on one feature at a time, runs tests, commits, and updates `claude-progress.txt`. Anthropic

Critical details from Anthropic's own research: use **JSON (not Markdown)** for feature lists because models are less likely to inappropriately edit JSON. Require **end-to-end testing, not just unit tests** — Claude tends to mark features complete without proper testing. Anthropic Use browser automation (Puppeteer MCP) for web app verification. The coding agent must **commit progress to git** with descriptive messages, creating a recoverable state after every completed feature. Anthropic

For Winston, this translates to:

```
json
```

```
// feature_list.json
{
  "features": [
    {
      "id": "feat-001",
      "name": "Add user settings endpoint",
      "service": "bos",
      "status": "failing",
      "test_command": "make test-backend",
      "deploy_command": "railway up",
      "smoke_test": "curl -s -o /dev/null -w '%{http_code}' https://bos.winston.app/api/settings"
    }
  ]
}
```

The agent flips `status` from `"failing"` to `"passing"` only after the smoke test succeeds. This creates an auditable, machine-readable record of what actually ran.

The "Waiting Kernel" prompt architecture that enforces bounded execution

The most battle-tested prompt pattern from the community (originally from r/PromptEngineering, validated by Cybercorsairs analysis) is the **"Waiting Kernel"** — a deterministic reasoning architecture with mandatory phases and hard stops:

```
OPERATING PRINCIPLES
1) Evidence before claims
  - If a fact depends on code/files: FIND → READ → then assert.
  - If unknown: label OPEN_QUESTION, propose safest default, move on.

2) Bounded execution
  - Work in deliverables (D1, D2, …) with explicit DONE checks.
  - After each deliverable: STOP. Do not continue to next deliverable.

3) Additive-first
  - Prefer additive changes over modifications.
  - Do not rename or restructure without explicit permission.

REASONING PIPELINE (Per request)
PHASE 0: FRAME  — Restate objective + constraints in 3-6 lines.
PHASE 1: PLAN   — Ordered checklist with a DONE check per step.
PHASE 2: VERIFY — FIND targets, READ exact sections, record discrepancies.
PHASE 3: EXECUTE — Make only the minimal change set.
PHASE 4: VALIDATE — Run build/tests once. If pass: deliverable DONE.
                    If fail: output error (last 30 lines) and STOP.

ANTI-LOOP RULES
- Never "keep going" after a deliverable.
- Never refactor to "make it cleaner."
- Never fix unrelated warnings.
- If baseline build/test is red: STOP and report; do not implement.
```

Phase 4 (VALIDATE) is mandatory — the agent cannot declare a deliverable complete without running tests. The anti-loop rules prevent scope creep, which is a secondary cause of execution failure: agents that try to do too much end up exhausting context before reaching the test/deploy phases.

For Winston's skill files, combine this with explicit service routing:

```markdown
```

```
# .claude/skills/feature-dev/SKILL.md
---
name: feature-development
description: Full feature development with mandatory test and deploy verification
allowed-tools: Read, Grep, Glob, Bash(*), Edit, Write
---

## Before Starting
1. Identify which service owns this feature (check Architecture Map in CLAUDE.md)
2. Confirm: "I will modify files ONLY in apps/{service}/"
3. Run existing tests to establish baseline: `make test-{service}`
4. If baseline is red: STOP and report. Do not proceed.

## Implementation Loop (repeat until VALIDATE passes)
PHASE 1: Write minimal code changes to files
PHASE 2: Run `make test-{service}` — paste full output
PHASE 3: If tests fail → read error → fix → return to PHASE 2
PHASE 4: If tests pass → run deploy command → paste output
PHASE 5: Run smoke test → paste HTTP status code

## Completion Criteria
Task is DONE only when ALL are true:
- [ ] Code written to correct service directory
- [ ] `make test-{service}` passes (output included)
- [ ] Deploy command executed (output included)
- [ ] Smoke test returns expected status code (output included)
```

Connecting your existing RAG pipeline thinking to agent eval

Your Google Drive document ("autonomouseprompt") reveals you're already thinking about versioned artifacts, eval pipelines, and feedback loops for a RAG system. The same mental model applies directly to agent skill evaluation: **treat skill files as versioned artifacts, eval results as regression gates, and failure modes as feedback into skill engineering**.

Anthropic's eval-driven development principles (January 2026) map cleanly:

- **Start with 20-50 tasks** sourced from real Winston feature failures
- **Every manual fix becomes a test case** — when you hand-fix what the agent should have done, encode that as an eval task

- **Graduate capability evals to regression suites** — once a skill achieves >90% pass rate on a set of tasks, those tasks become regression tests that must stay green
- **Read transcripts** when scores don't climb — verify it's agent performance, not eval bugs (Anthropic)

The feedback loop structure is: define eval suite → run agent on tasks (containerized) → grade with multi-checkpoint rubric → analyze which checkpoints fail most → add explicit skill instructions targeting those failures → re-run evals → compare. (Openai) (OpenAI Cookbook) The Anthropic principle that applies most: **if agents skip tests, don't fix it with better planning prompts — fix it with hooks that deterministically run tests after every edit**. (humanlayer)

Concrete next steps ranked by expected impact

The patterns that will move your 58.5% pass rate most are, in order of expected impact:

PostToolUse hooks to auto-run `make test-backend` and `make test-frontend` after every file edit — this is deterministic and cannot be skipped, (Claude) matching Aider's `--auto-test` pattern that drives their strongest retry improvement rates. Configure in `.claude/settings.json` immediately.

Multi-checkpoint eval rubric that separately scores code writing, test invocation, test passing, deployment, and smoke testing — this will reveal exactly which phase your agent fails at and give you targeted optimization data instead of a single aggregate number.

State machine skill files with explicit phase gates preventing transition from "implementing" to "complete" without passing through "testing" and "deploying" — this mirrors the QuantumBlack finding that rule-based workflow engines outperform self-regulation for phase sequencing. (Medium)

The two-agent initializer/coder pattern with a JSON feature list as source of truth — Anthropic's own recommendation for long-running tasks, preventing premature completion declarations. (Anthropic)

Permission configuration allowing `Bash(make *)`, `Bash(railway *)`, `Bash(git push *)`, and `Bash(curl *)` in `.claude/settings.json` to remove friction from execution — if the agent has to ask permission for every test run, it's more likely to skip them.

The compound effect of these changes should be substantial. Aider's data shows that the delta between first-attempt and retry-with-feedback pass rates is typically **20+ percentage points** (GitHub) — and that's from a single mechanism (auto-test). Layering hooks, state gates, proof-of-execution requirements, and multi-checkpoint eval should close far more of the gap than skill content improvements alone.